

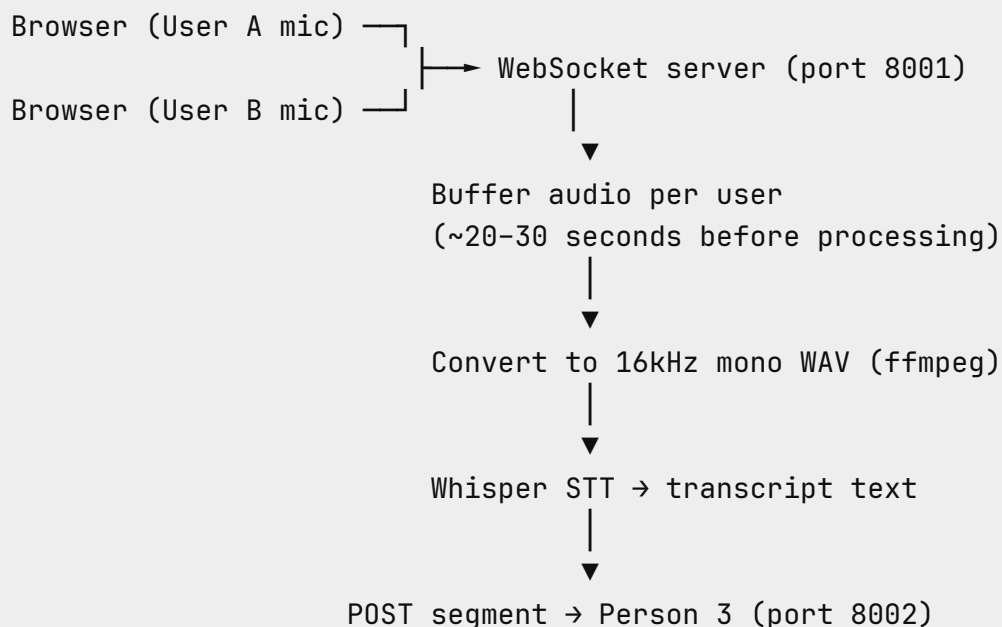
Audio_Pipeline

Real-Time Audio Pipeline (Whisper STT)

Role

You are the ears of the system. Raw audio comes in from both users' browsers and you output clean, timestamped, speaker-labeled transcript segments. Everything downstream – sliding windows, drift detection, warnings – depends entirely on your output. Nothing works without you.

Data Flow



What You Build

1. WebSocket Server – Port 8001

Accept connections at: `ws://localhost:8001/audio/{barter_id}/{user_id}`

- `barter_id` – which session this audio belongs to

- `user_id` – 1 (Alice) or 2 (Bob), used as speaker label

The browser sends binary audio chunks every 5 seconds via `MediaRecorder`. You receive these and buffer them before transcription.

2. Audio Buffering

Do not run Whisper on every 5-second chunk – too fragmented. Accumulate chunks per `(barter_id, user_id)` pair until you have approximately 20–30 seconds of audio, then transcribe the batch.

Buffer key: `(barter_id, user_id)` – keeps two simultaneous users' audio separate.

3. Audio Conversion

The browser's `MediaRecorder` outputs `.webm` by default. Whisper requires `.wav` at 16kHz mono. Convert using `ffmpeg` before passing to Whisper.

Conversion settings: 16kHz sample rate, mono channel.

4. Whisper Transcription

Use the `openai-whisper` Python library (local, offline, free). Recommended model: `base`. Use `tiny` if speed is a concern.

After transcription, attach:

- `speaker` label (`user_1` or `user_2`)
- `timestamp_start` – when the audio chunk started
- `timestamp_end` – when transcription completed

Discard empty transcriptions (silence). Do not forward blank segments.

Delete temp audio files after transcription to avoid disk buildup.

5. Session End Endpoint

POST `/session/{barter_id}/end`

Called by Person 1 when the session completes. Flush any remaining buffered audio for that session, transcribe it, forward the final segment to Person 3, then notify Person 3 the session has ended.

Output Format – Sent to Person 3

Every transcript segment is POSTed to `http://localhost:8002/ingest/segment`:

```
{
  "barter_id": 1,
  "speaker": "user_1",
  "text": "Today we will cover basic addition and subtraction using whole numbers.",
  "timestamp_start": 1710234000.0,
  "timestamp_end": 1710234028.5
}
```

Field	Type	Description
barter_id	int	Which session
speaker	string	"user_1" or "user_2"
text	string	Transcribed text from Whisper
timestamp_start	float	Unix timestamp – chunk start
timestamp_end	float	Unix timestamp – transcription done

Interface Contracts

You Receive (inbound)

From	How	What
Person 1 (frontend)	WebSocket binary frames	Raw audio chunks
Person 1 (backend)	POST /session/{barter_id}/end	Session end signal

You Call (outbound)

Target	When	Endpoint
Person 3	After each transcription	POST http://localhost:8002/ingest/segment
Person 3	After flushing final buffer	POST http://localhost:8002/session/{barter_id}/end

What You Do NOT Build

- Sentence-BERT embeddings or similarity scoring → Person 3
- Sliding window logic → Person 3
- Warning decisions → Person 4
- Frontend mic capture → Person 1 (they send you the audio)
- QA verdict or trust updates → Person 1

Libraries

```
fastapi, uvicorn, websockets, openai-whisper, ffmpeg (system install),  
httpx, torch
```

Definition of Done

- ☐ WebSocket server runs on port 8001
- ☐ Two simultaneous users stream without interfering with each other
- ☐ Audio is buffered (~20–30s) before transcription – not per 5s chunk
- ☐ ffmpeg converts WebM to 16kHz mono WAV without errors
- ☐ Whisper `base` model transcribes English speech correctly
- ☐ Empty/silent chunks are discarded and not forwarded
- ☐ Each segment is POSTed to Person 3 within ~5 seconds of transcription
- ☐ Session end flushes remaining buffer and notifies Person 3
- ☐ Temp audio files are deleted after each transcription